

Topological Sorting

Topological sorting is the problem of arranging vertices of a directed acyclic graph (DAG) in a linear order such that for every directed edge (u, v) , vertex u comes before v in the order.

Natural Explanation

If vertices represent tasks and edges represent “must-do-before” constraints, topological sort gives a valid execution order: all prerequisites come before dependent tasks.

Existence and Uniqueness

A topological sort exists **if and only if** the graph is a DAG (acyclic).

If the graph has a cycle, no valid ordering exists (you cannot satisfy all constraints simultaneously).

A topological sort is **not unique** unless the DAG has a specific structure (e.g., is a path).

Algorithm via DFS

One common approach uses [[Depth-First Search|DFS]]:

Idea: finish times (when DFS-VISIT returns) are in reverse topological order.

```
TOPOSORT-DFS(graph)
    visited = empty set
    finish_order = empty list

    for each vertex v in graph.VERTICES():
        if v not in visited:
            DFS-VISIT-TOPO(v, visited, finish_order)

    reverse(finish_order)
    return finish_order

DFS-VISIT-TOPO(vertex, visited, finish_order)
    add vertex to visited
    for each neighbor in graph.NEIGHBORS(vertex):
        if neighbor not in visited:
```

```
DFS-VISIT-TOP0(neighbor, visited, finish_order)
    append vertex to finish_order
```

Time: $O(|V| + |E|)$ — one DFS pass

Algorithm via Indegree (Kahn's Algorithm)

Alternative approach using indegrees:

Idea: vertices with indegree 0 have no incoming constraints, so they can be processed first. Remove them and update indegrees of neighbors.

```
TOPOSORT-INDEGREE(graph)
    indegree = compute indegrees for all vertices
    queue = empty queue

    for each vertex v:
        if indegree[v] == 0:
            queue.ENQUEUE(v)

    order = empty list
    while not queue.ISEMPTY():
        vertex = queue.DEQUEUE()
        append vertex to order
        for each neighbor in graph.NEIGHBORS(vertex):
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.ENQUEUE(neighbor)

    return order
```

Time: $O(|V| + |E|)$ — process each vertex and edge once

Cycle Detection

If you use the indegree algorithm and the final order has fewer than $|V|$ vertices, a cycle exists.

If you use DFS and encounter a back edge (edge to an ancestor), a cycle exists.

Properties

- **Linear:** output is a single sequence
- **Respects edges:** if (u, v) exists, u comes before v

- **Not unique:** many valid topologies may exist
-

Use Cases

- **Task scheduling:** with dependencies or prerequisites
 - **Build systems:** compiling libraries and executables in correct order
 - **Dependency resolution:** installing packages in correct order
 - **Course prerequisites:** determining valid course sequences
-

Related

- [\[\[Depth-First Search\]\]](#)
- [\[\[Graphs\]\]](#)
- [\[\[Shortest Paths on DAGS\]\]](#)